다이캐스팅 불량유형, 금형을 열기 전에 알 수 있는가

3조 최종 보고서 · 강동하 · 구정희 · 이지수 · 이창우 · 최종 모델: KNN (k=15 · 맨해튼 거리 · 거리 가중 · MinMax)

요약

- 샷마다 남는 공정 기록 21개 값으로 정상 + 불량 6종, 모두 7가지를 예측했다. 최종 모델은 KNN이고 public 리더보드 macro F1은 **0.5890**이다.
- 이 점수의 큰 부분은 **21개 값이 모두 같은 중복 기록** 덕분이다. train의 58.7%, test의 57.7%가 train에 중복 기록을 가진다. 학습 쪽에 같은 기록이 있는 샷은 불량의 92%를 잡지만, 없는 샷은 13%만 잡는다.
- 과거로 학습해 다음 시기를 맞히는 **시간순 검증**에서는 macro F1 0.138, 불량 231건 중 45건(19%)만 잡았다. 현장에서는 앞으로 새로 만들 샷을 판정해야 하므로, 실제로 썼을 때의 성능은 이 시간순 결과에 가깝다.
- 그래서 **지금 하는 검사는 그대로 두고, 모델이 불량으로 의심한 샷부터 먼저 검사하는 방식**으로 몇 주 시험해 볼 것을 제안한다. 그 기간에 실제로 몇 개를 맞혔는지 세어 본 뒤, 검사를 줄여도 되는지 판단한다.
- 같은 시기의 기록을 5배로 늘려도 시간순 성능은 그대로였다(0.136 → 0.141). 금형 온도처럼 지금 재지 않는 값을 새로 기록해야 한다.

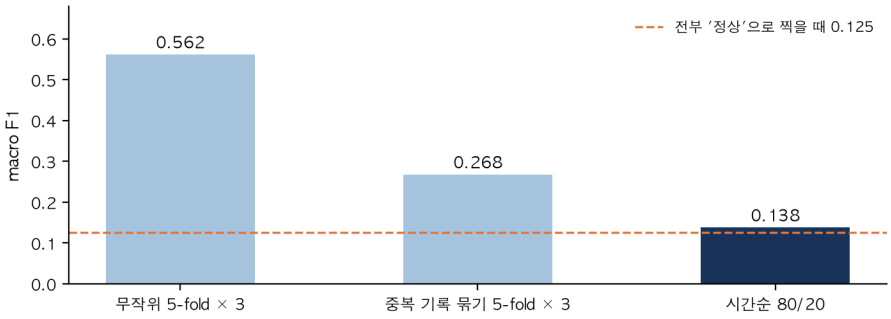


그림 1. 같은 KNN이라도 검증 방식에 따라 macro F1이 0.562 → 0.268 → 0.138로 떨어진다

이 보고서에서 쓰는 말

말	뜻
중복 기록	21개 입력값이 모두 같은 샷. 생산 순서상 바로 옆 행끼리 생김. 같은 샷의 이중 기록이거나 설정이 같은 연속 샷으로 추정
macro F1	7개 클래스 F1의 단순 평균. 검증 구간에 없는 클래스도 0점으로 넣어 항상 7로 나눔. 정상만 많이 맞혀서는 오르지 않음
무작위 검증	train을 무작위로 나눠 검증. 중복 기록이 학습·검증에 나뉘어 들어가 리더보드와 조건이 비슷함
중복 기록 묶기 (group CV)	값이 같은 샷을 같은 묶음에 넣고 나눠 검증. 처음 보는 샷에 대한 성능
시간순	shot_id 순서로 앞 80%로 학습하고 뒤 20%로 검증. 현장 배포 조건에 가장 가까움
놓침 · 헛경보	불량을 정상이라고 한 것 · 정상을 불량이라고 한 것

데이터: train 5,274샷(정답 있음), public 1,130샷(리더보드), private 1,131샷(최종일 채점, 아직 결과 없음). 모든 분석·검증·운영표는 train으로 만들었다. test(public + private)는 정답 없이 입력값만 보고 "같은 문제가 test에도 있는가"만 확인했다.

1. 데이터에서 찾아낸 문제와 처리, 그 근거

문제	근거 (관찰한 사실)	해석 · 처리
중복 기록	train 5,274샷 중 3,097샷(58.7%)이 다른 train 샷과 21개 값이 모두 같음. 모두 생산 순서상 바로 옆 행. 57개 그룹은 값이 같은데 정답이 다름. test의 57.7%도 train에 중복 기록 있음	**추정**: 같은 샷의 이중 기록이거나 설정이 같은 연속 샷 (데이터로는 구분 불가). 행은 유지하고, 중복 기록 묶기와 시간순으로 '처음 보는 샷' 성능을 따로 측정 (6장)
제품 · 시기별 차이	제품 1(2,950샷) → 제품 2(2,324샷) 순서로 생산, 제품별 불량 유형이 다름. 제품 1 불량률은 초반 37% → 후반 1%로, Spray_1_Time 설정(1.0 → 0.8 → 1.2)과 함께 바뀜	Product_Type은 특징으로 유지. shot_id는 규칙상 특징으로 쓰지 않고 정렬에만 사용
클래스 불균형	정상 77.6%(4,092샷), 기타불량 1.4%(76샷)	데이터 수는 그대로 두고 층화 분할로 검증. 최종 KNN에는 클래스 가중치 없음. 불균형 대응을 6가지 시도했지만 macro F1을 올린 방법은 없음 (아래 표)
0 값 · 긴 사이클	train: 0 값 6행은 모두 정상, 40초 넘는 사이클 15행 중 14행이 정상. test에도 0 값 2행, 긴 사이클 3행	불량 신호가 아니고 test에도 있어 삭제하지 않음
공장 온 · 습도 결측	train 63행 · test 27행. 두 연속 구간에서 공장 관련 6개 열이 함께 비어 있음. 결측 구간 불량률 23.8% ≈ 전체 22.4%	**추정**: 기록 장치가 멈춘 구간. 최종 KNN은 학습 데이터의 중앙값으로 채움. 중앙값 · 직전 값 · 선형 보간의 macro F1 차이는 ±0.001 이내
쓸 수 없는 열 11개	식별 열 3개(shot_id · id · Shot)는 규칙상 금지. 관리범위(Min/Max) 열 8개는 모든 행이 같은 값	모두 제외 → **특징 21개**

최종 학습과 제출

k · 거리 등 설정은 train 안의 교차검증으로 정했다(3장). 최종 모델은 **train 5,274샷 전체**로 다시 학습했고, 결측 채우기와 스케일링도 train으로만 계산했다. 행은 하나도 지우지 않았다. 이 모델로 **public 1,130샷 + private 1,131샷**을 예측해 제출했다. public macro F1은 0.5890이고, private은 아직 채점 전이다. 재현: 학생배포/ 안에서 `python main.py`. 다시 실행해 제출 파일과 비교했고, 2,261샷 중 예측이 다른 샷은 0개였다.

불균형 대응 시도와 결과 — 불량을 더 잡게는 했지만 macro F1은 오르지 않았다

방법	적용 모델	결과	판정
SMOTE · 노이즈로 불량 샘플 늘리기	KNN	탐지율 48% → 62%, macro F1 −0.010 ~ −0.023	미사용
기타불량 · 변형만 늘리거나 확률 키우기	KNN	macro F1 −0.003 ~ −0.014, 기타불량 F1 0.25 → 0.19	미사용
정상 확률을 낮춰 불량 쪽으로 판정	KNN+RF+XGB 앙상블	정상 배율 0.9 ~ 0.4 모두 macro F1 하락	미사용 (배율 1.0)
클래스 가중치 + 판정 가중치	RF+ET+XGB 앙상블	처음엔 0.58로 보였지만 중첩 검증으로 다시 재니 0.546 (+0.006뿐)	미사용
손실 클래스 가중치	작은 MLP	탐지율 3% → 97%로 거의 다 불량이라 함. 시간순 macro F1 0.125 → 0.074	비교용으로만
손실 클래스 가중치	로지스틱	macro F1 0.131 → 0.235로 올랐지만 정상 4,092건 중 3,147건에 경보 (3장)	비교용으로만
불량을 더 잡도록 바꾼 제출	리더보드	탐지율 71.5%, 헛경보 97건 → public 0.5643 (KNN은 탐지율 56.1%, 헛경보 28건 → 0.5890)	최종 제출에서 제외

불균형을 맞추면 불량을 더 많이 '의심'하게 되지만, 그만큼 정상품 경보가 늘어 7가지를 고르게 맞히는 macro F1은 오르지 않았다. 원래 불량을 거의 못 잡던 로지스틱만 예외였다. 그래서 최종 KNN은 데이터를 그대로 두고 가중치도 쓰지 않았다.

2. 설계 카드 — 입력 · 구조 · 활성화 · 손실 · 학습률 · epoch를 왜 그렇게 정했는지

MLP 비교 대상은 **PLR MLP**(숫자 변수를 주기 함수로 임베딩한 뒤 MLP에 넣는 구조)로 정했다.

항목	선택	이유 (이 데이터 기준)
입력	특징 21개 (베이스라인 · KNN과 같은 입력). 결측 중앙값 → 표준화, 둘 다 학습 데이터로만 계산	변수마다 단위가 다름 (속도 0.1대, 주조압 1,000대)
입력 임베딩	PLR: 변수마다 sin · cos 16쌍(주파수 학습, σ=0.3) → Linear(32→16) + ReLU	불량률이 설정값에 따라 계단처럼 바뀜 (Spray_1_Time 1.0 → 33%, 1.2 → 1%). 일반 MLP로는 좁은 구간 차이를 잡기 어렵다고 판단
구조	21×16=336 → 512 → 256 → 7	임베딩 뒤 336차원이라 첫 층을 넓게 두고 7개 클래스로 줄임
활성화	ReLU	표준적이고 학습이 안정적
손실	CrossEntropy (클래스 가중치 없음)	작은 MLP(4장 B①)에서 가중치를 주면 불량 예측이 지나치게 늘어남. PLR MLP에서 가중치 유무는 직접 비교하지 않음
학습률	AdamW 0.002, OneCycle 스케줄	처음에 크게 움직이고 끝에서 줄임
epoch	300, 배치 128, 조기 종료 없음	비교 기준을 고정하기 위해. PLR MLP의 epoch 곡선은 그리지 않음 (5장 곡선은 작은 MLP)
시드	random · numpy · torch 고정, 스레드 1개	재현

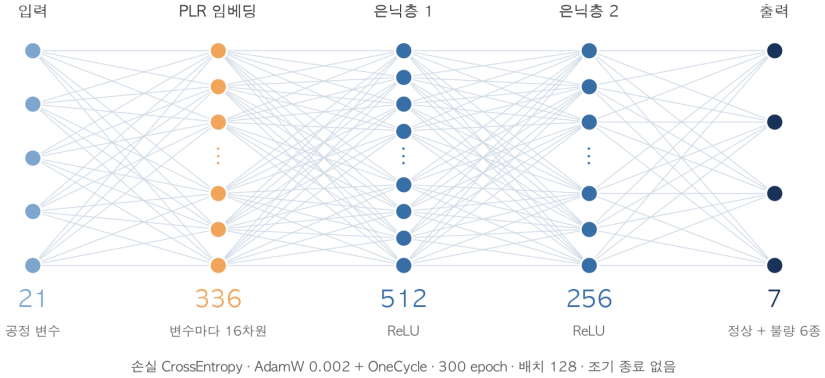


그림 2. PLR MLP 구조 — 입력 21개를 변수마다 16차원으로 펼친 뒤(336) 512 → 256 → 7로 줄인다

3. 베이스라인 대조 — 로지스틱 / RF vs MLP, 결과의 이유

같은 5-fold(시드 7)로 모든 train 샷을 한 번씩 검증한 결과(7개 클래스 macro F1).

모델	macro F1	실제 불량 1,182건 중 '불량'이라고 한 수	실제 정상 4,092건 중 '불량'이라고 한 수	'불량'이라고 예측한 샷 (5,274샷 중)
로지스틱 회귀	0.1306	5건 (0.4%)	10건	15샷
로지스틱 (class_weight)	0.2348	1,128건 (95.4%)	3,147건	4,275샷
RandomForest	0.5714	598건 (50.6%)	124건	722샷
KNN (최종)	**0.5825**	**600건 (50.8%)**	**120건**	**720샷**
PLR MLP	0.5342	-	-	-

리더보드 공개 기준선: 로지스틱 0.1313 · 로지스틱(class_weight) 0.2517. KNN 설정은 k(5~31) · 거리(p=1, 2) · 가중(distance, uniform) · 스케일러(MinMax, Standard, Robust) 84개 조합을 5-fold × 3회(시드 42)로 비교해 1등(0.5551)인 조합이다. k는 11~21 사이면 차이가 0.01 미만이었다. 같은 train으로 고른 설정이라 위 0.58은 약간 낙관적일 수 있다.

- **로지스틱은 직선 경계라 실패했다.** 가중치 없이는 거의 전부 정상, 가중치를 주면 거의 전부 불량으로 예측했다. 불량은 "Spray_1_Time 1.0이고 덥고 건조할 때"처럼 조건이 겹친 좁은 구간에 있다.
- **KNN이 MLP보다 높은 이유 (추정)**: 거리 가중 KNN은 값이 똑같은 학습 샷(거리 0)의 정답을 그대로 가져온다. 학습 쪽에 중복 기록이 있는 샷은 macro F1 0.83, 없는 샷은 0.27이었다(6장).
- **리더보드에서도 MLP가 낮았다.** 제출한 PLR MLP(이슬점 · 건조도 등 변수를 더한 버전)는 public 0.5465(탐지율 62.5%)로 KNN 0.5890(56.1%)보다 낮았다. 불량은 더 잡았지만 헛경보가 늘었다.

4. 비교 실험 3종 — 한 번에 하나만 바꿔서

작은 MLP (64-32)에서 하나씩 바꿔 본 세 가지 — 시드 3개 평균

기본: 클래스 가중치 있음, Dropout 0, Adam 0.001, 특징 22개(결측 표시 열 포함). 괄호는 불량 탐지율.

실험	설정	무작위 80/20	시간순 80/20
① 손실 클래스 가중치	있음 (기본)	0.210 (97%)	0.074 (64%)
	없음	0.157 (3%)	0.125 (0%, 전부 정상)
② 은닉층 크기	32-16	0.229 (96%)	0.045 (91%)
	128-64	0.235 (96%)	0.084 (59%)
③ 입력 임베딩	PLR σ=0.1	**0.252** (95%)	**0.116** (37%)
	PLR σ=1	0.238 (93%)	0.039 (97%)

- ① 가중치가 있으면 불량을 거의 다 의심해(탐지율 97%) 헛경보가 넘친다. 가중치를 빼면 시간순에서 전부 정상으로 예측해 0.125(= 정상 F1 0.875 ÷ 7)가 나온다. 시간순 점수는 반드시 탐지율과 함께 봐야 한다.
- ② 은닉층이 클수록 조금 나아졌지만 모두 0.24 이하.
- ③ PLR(σ=0.1)이 작은 MLP 중 가장 나았지만 KNN(무작위 0.568, 시간순 0.141)보다 낮다. 그래서 PLR 구조를 키운 PLR MLP(2장)를 비교 대상으로 삼았다.

5. 학습곡선 두 가지 — (a) epoch 곡선 (b) 데이터양 곡선

작은 MLP(64-32) · KNN · RF로 그렸다 (PLR MLP 결과 아님).

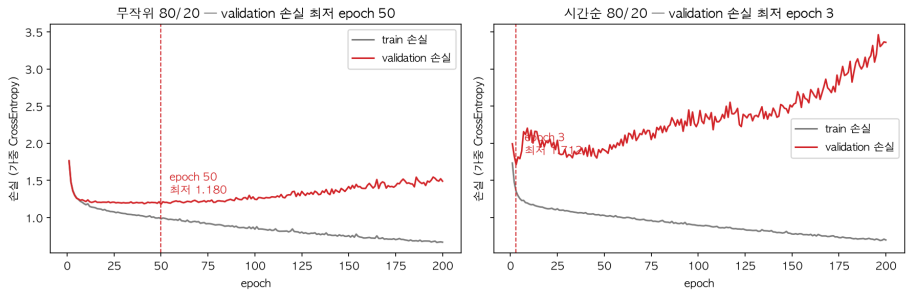


그림 3. (a) 작은 MLP의 epoch 곡선 (클래스 가중 손실, 조기 종료 없이 200 epoch)

- **무작위 80/20**: 검증 손실 최저는 epoch 50(1.18). train 손실이 0.67까지 내려가는 동안 검증 손실은 1.49로 올랐다 → 50 epoch부터 과적합.
- **시간순 80/20**: epoch 3(1.71)에서 최저, 바로 올라 200 epoch에 3.36 → 학습할수록 다른 시기는 더 틀린다.

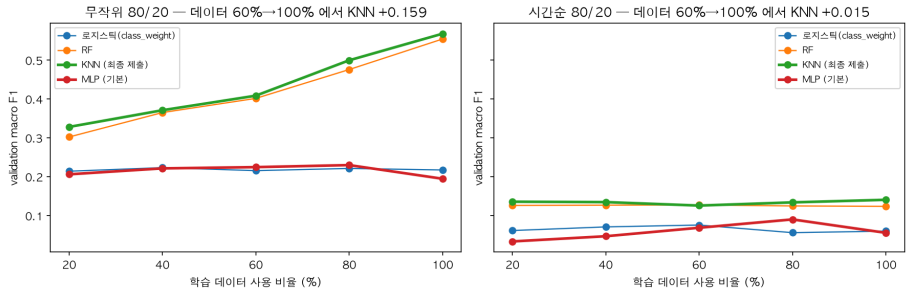


그림 4. (b) 학습 데이터 20~100% 사용 시 검증 macro F1 (검증 20% 고정)

분할 · 모델	20%	40%	60%	80%	100%
무작위 · KNN	0.328	0.371	0.408	0.499	**0.568**
무작위 · RF	0.302	0.365	0.401	0.476	0.554
무작위 · MLP (64-32)	0.206	0.221	0.225	0.230	0.195
시간순 · KNN	0.136	0.135	0.126	0.134	0.141
시간순 · RF	0.126	0.127	0.127	0.125	0.124
시간순 · MLP (64-32)	0.033	0.047	0.069	0.090	0.056

- 무작위 분할에서 KNN · RF가 데이터가 늘수록 오르는 것은, 검증 샷의 중복 기록이 학습 쪽에 들어갈 확률도 함께 높아진 영향이 섞여 있을 것으로 **추정**한다.
- **데이터를 더 모을 것인가?** 시간순에서는 학습 데이터를 5배로 늘려도 KNN 0.13~0.14, RF 0.12~0.13에서 거의 그대로다. → **같은 기간의 기록을 더 늘리는 것만으로는** 다른 시기 예측이 좋아지지 않는다. 금형 온도 같은 새 측정값이 도움이 될지는 이 실험으로 확인되지 않았다.

6. 분할 바꿔 재보기 — 무작위 vs 시간순, 어느 쪽이 배포 성능인가

최종 KNN을 train 5,274샷 안에서 두 방법으로 나눠, 한쪽으로 학습하고 다른 쪽으로 검증했다.

	무작위 80/20	시간순 80/20 (shot_id 앞 80% → 뒤 20%)
검증에 있는 클래스	7개	**3개** (정상 · 미충전 · 기공)
macro F1 (7개 평균)	**0.570**	**0.138**
불량 탐지	237건 중 126건 (53%)	231건 중 45건 (19%)
유형까지 맞힘	117건	18건
헛경보	정상 818건 중 27건	정상 824건 중 155건
검증 샷 중 학습 쪽에 중복 기록이 있는 비율	48%	0.1%

중복 기록 효과를 따로 재 본 결과 (5-fold 시드 7, 검증 결과를 둘로 나눠 채점)

	샷	macro F1	불량 탐지
학습 쪽에 중복 기록 있음	2,500	0.831	563건 중 517건 (92%)
중복 기록 없음	2,774	0.269	619건 중 83건 (13%)
무작위 5-fold × 3 / 중복 기록 묶기 5-fold × 3	-	0.562 / 0.268	-

두 그룹은 중복 기록 유무 말고도 시기 · 유형 구성이 다를 수 있어 차이 전부가 중복 기록 때문이라고 할 수는 없다. 다만 점수의 큰 부분이 학습 데이터에 같은 값이 있는 샷에서 나온다는 것은 확인됐다.

데이터양을 늘렸을 때도 두 분할은 다르게 움직였다(5장 그림 4). 무작위 분할은 학습 데이터 20% → 100%에서 0.328 → 0.568로 올랐지만, 시간순은 0.136 → 0.141로 거의 그대로였다.

어느 쪽이 배포 성능인가 → 시간순

- 무작위 분할은 리더보드와 조건이 비슷하다. public · private은 생산 기간 전체에 고르게 퍼져 있고(10구간마다 101~133샷) 57.7%가 train에 중복 기록을 가진다. 무작위 0.570 ≈ public 0.589.
- 현장에서 판정할 샷은 아직 생산되지 않은 샷이다. 학습 데이터에 값이 똑같은 샷이 있다는 보장이 없다.
- 시기마다 불량 구성이 바뀐다. 변형 106건은 모두 생산 초반 1/5 구간에, 얼룩 98건은 4구간에만 있다.
- 다른 모델도 시간순에서는 모두 낮았다: RF 0.124(탐지 4%), XGB 0.126(3%), 로지스틱(class_weight) 0.060(86%).

→ 리더보드 점수(0.589)는 무작위 분할 조건의 점수이고, 처음 보는 시기에서는 불량 231건 중 45건(19%)을 잡는 수준이다.

7. 운영 기준 설계 — 비용표, 결정 기준별 놓침 · 헛경보 · 총비용, 고른 운영안

오진	현장에서 생기는 일	비용 (단위)
놓침 (불량 → 정상)	불량이 후공정 · 출하로 넘어감 → 재작업, 반품, 고객 클레임	10
헛경보 (정상 → 불량)	정상품을 한 번 더 검사	1

- **비용 = 놓침 × 10 + 헛경보 × 1. 비용 단위 1 = 정상품 1개를 한 번 더 검사하는 비용**이다(원화 단가가 아닌 상대값). 예: 재검사 1건이 1,000원이면 비용 781은 하루 약 78만 1천 원이다. 10 : 1은 가정이며 현장 단가로 정해야 한다.
- 포함하지 않은 비용: 탐지한 불량의 검사 · 폐기 · 재작업, 유형 오분류로 조치가 틀리는 비용, 검사자 자신의 놓침, 모델 운영 비용. **검사 인력 부담 = 전체 경보 수**로 따로 센다.
- 하루 생산량 1,000샷은 가정이다(사이클 중앙값 22.6초 → 8시간 약 1,270샷에서 대기 시간을 뺀 값). 검증 1,055샷 결과를 1,000샷으로 환산했다(반올림 때문에 탐지 + 놓침이 1 차이 날 수 있음).
- **결정 기준: 정상 확률에 곱하는 배율**(작을수록 불량으로 더 판정). 각 배율을 같은 검증 데이터에 적용한 결과라 고른 배율의 수치는 독립 검증값이 아니다. 운영표는 모두 train에서 떼어 둔 검증 데이터로 만들었고 public 점수는 쓰지 않았다.

(1) 배포 판단 기준 — 시간순 80/20 (처음 보는 시기), 하루 1,000샷 · 불량 219샷

기준	탐지 (유형 정답)	놓침	헛경보	전체 경보	하루 비용 (재검사 1건 = 1)	손익분기 비율*
배율 1.0 (제출 기준)	43 (17)	176	147	190	1,907	3.6
배율 0.5	97 (42)	122	341	438	1,561	3.6
배율 0.3	135 (64)	84	474	609	1,314	3.7
배율 0.15	179 (85)	40	633	812	1,033	3.7
검사 안 함	0	219	0	0	2,190	-
전수 검사	**219**	**0 †**	**781**	**1,000**	**781**	-

* 놓침 1건 비용이 헛경보의 이 배수보다 작아야 전수 검사보다 싸다. † 검사가 불량을 하나도 놓치지 않는다는 가정.

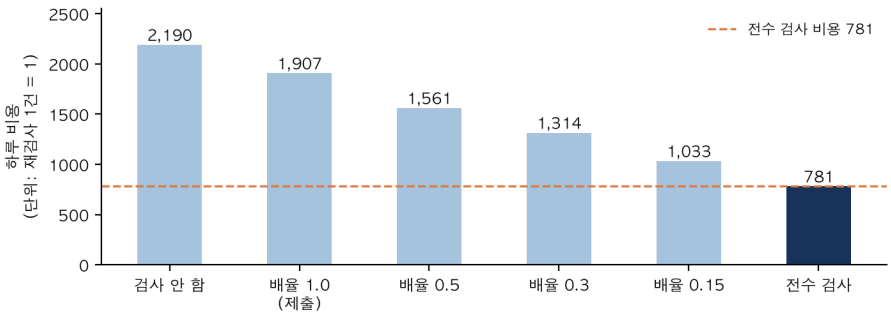


그림 5. 처음 보는 시기 기준 하루 비용 (단위: 재검사 1건 = 1, 놓침 1건 = 10)

참고 — 무작위 80/20 기준(중복 기록이 많은 조건, 배포 성능 아님): 배율 1.0에서 하루 놓침 105 · 헛경보 26 · 비용 1,076, 배율 0.3에서 놓침 58 · 헛경보 127 · 비용 707, 배율 0.15에서 놓침 30 · 헛경보 221 · 비용 521로 전수 검사(775)보다 싸 보인다. 하지만 이미 답을 본 문제와 같은 조건이라 운영 판단에는 쓰지 않는다.

고른 운영안

- **비용 최소 기준 (10 : 1, 시간순)**: 모든 배율이 전수 검사(781)보다 비싸다(1,033~1,907). 놓침 비용이 헛경보의 약 3.6배 이하일 때만 모델 판정이 전수 검사보다 싸다.
- **검사 인력 기준**: 하루 검사 가능 수량은 받은 자료에 없다. 수량이 정해지면 전체 경보가 그 안에 드는 배율을 고르면 된다(예: 하루 200샷 → 배율 1.0, 경보 190).
- **제안: 검사는 그대로, 순서만 모델로.** ① 지금 하는 검사는 줄이지 않고 그대로 다 한다. ② 모델이 불량으로 의심한 샷을 먼저 검사한다. ③ 몇 주 동안 의심한 샷 중 실제 불량 수와, 표시 없이 지나간 불량 수를 센다. ④ 그 결과와 현장이 정한 비용 비율 · 하루 검사 가능 수량을 보고, 모델 표시를 계속 쓸지 · 검사를 줄여도 될지 · 배율을 얼마로 할지 정한다.
- 리더보드는 macro F1이 기준이라 배율 1.0으로 제출했다.

8. 불량 원인 분석 — 불량유형별 구간 불량률, 현장 제안과 한계

모델(RF, 공정 변수 포함)의 중요도 상위: 이슬점, High_Velocity, 건조도, 공장 습도, Spray_Time, Cycle_Time, 냉각수 온도 · 압력. 중요도는 "예측에 쓸모 있다"는 뜻이지 원인이라는 뜻이 아니다. 이 조건으로 구간을 나눠 불량률을 직접 셌다(train, 중복 기록도 각각 한 샷으로 셈).

(1) 변형 (Deformation, 제품 1)

106건 중 101건이 Spray_1_Time 1.0 설정 기간에 나왔다.

조건 (Spray_1_Time 1.0 기간 안)	샷	변형 비율
기간 전체	988	10.2%
+ 공장 온도 > 34.7℃	536	15.7%
+ 용탕로 온도 ≤ 700℃	268	21.6%
+ 이형제 분사 시간 ≤ 8.0초	**197**	**25.9%**
공장 온도 ≤ 32.3℃	217	0.0%

건조도 중앙값: 변형 12.5 · 기타불량 11.9 vs 정상 7.9 → 덥고 건조한 날 변형이 많이 관찰됐다. 기준값은 데이터를 보고 고른 값이라 다른 기간에서도 같은 차이가 나는지는 확인이 필요하다.

(2) 기공 · (3) 얼룩 (제품 2)

조건	샷	비율
기공 · 냉각수 온도 ≤ 26.4℃	512	**23.4%**
기공 · 냉각수 온도 > 26.4℃	1,812	6.2%
얼룩 · 냉각수 압력 하위 1/3 (≤ 2.640)	852	0.0%
얼룩 · 냉각수 압력 상위 1/3 (> 2.730)	762	**9.7%**

생산 시기와 겹치는지 (제품 2를 생산 순서 4구간으로)

시기	기공: 냉각수 온도 낮음 / 높음	얼룩: 압력 낮음 / 높음
1/4	14.5% (282) / 1.0% (299)	- (0) / 0.0% (134)
2/4	34.3% (230) / 8.3% (351)	- (0) / 12.2% (441)
3/4	- (0) / 5.5%	0.0% (291) / 10.7% (187)
4/4	- (0) / 8.4%	0.0% (561) / - (0)

- 냉각수 온도가 낮은 샷은 제품 2의 앞 절반에만, 냉각수 압력이 낮은 샷은 뒤 절반에만 있다. 즉 **조건과 생산 시기가 겹친다.** 같은 시기 안에서도 차이는 남았지만(기공 2개 구간, 얼룩 1개 구간), 그 시기에 함께 바뀐 다른 조건의 영향은 배제하지 못했다.
- **현장 제안**: 제품 2는 냉각수 온도를 26.4℃ 넘게 관리하면 기공이 23.4% → 6.2% 수준으로 줄어들 **가능성**이 있다. 제품 1 변형은 Spray_1_Time 1.0 기간 중 덥고(공장 34.7℃ 초과) 용탕로 온도가 낮을(700℃ 이하) 때 집중되므로 이 조합을 우선 점검한다.
- **한계**: 냉각수 온도 · 압력과 기공 · 얼룩은 관련성이 관찰된 것이지 원인이 확인된 것이 아니다. 조건과 생산 시기가 겹쳐 개선 효과는 아직 확인되지 않았다.
- **확인 방법**: 다른 조건은 그대로 두고 한 조건만 바꾼 통제 시험 생산(예: 같은 날 냉각수 온도만 26℃ / 27℃로 나눠 생산하고 기공 비율 비교). 변형도 Spray_1_Time 1.0 설정에서 용탕로 온도 · 분사 시간을 바꿔 같은 방식으로 확인한다.

9. 현장 언어 번역 — 건수와 분모로 말하기

최종 KNN, 5-fold(시드 7, 3장과 같은 fold). 모든 train 샷을 그 샷을 학습에 쓰지 않은 모델로 예측했다.

불량	실제	유형까지 맞힘	다른 불량으로	정상으로 놓침	이 유형 경보	그중 헛경보	그중 유형 오분류
미충전	469	240	14	215	298	45	13
기공	233	144	2	87	169	23	2
박리	198	75	11	112	108	23	10
얼룩	100	69	6	25	77	6	2
변형	106	26	2	78	51	18	7
기타	76	10	1	65	17	5	2
합계	**1,182**	**564**	**36**	**582**	**720**	**120**	**36**

- 불량 1,182건 중 600건을 불량으로 잡았고, 그중 564건은 유형까지 맞혔다. 36건은 다른 불량 유형으로 알렸다.
- 놓친 582건은 모두 '정상'으로 판정된 것이다. 정상 4,092건 중 120건을 불량이라고 잘못 경보했다. → 약점은 유형 구분보다 **불량인지 아닌지 가리기**다.
- 미충전을 기공으로 착각한 것은 2건, 기공을 미충전으로 착각한 것은 2건이다(둘은 조치가 다르므로 따로 셌다).
- 변형은 106건 중 26건만 맞혔고(같은 기록 비율 9%로 가장 낮음), 변형이라고 알린 51건 중 18건은 정상, 7건은 다른 불량이었다.

- 위 수치는 중복 기록이 섞인 조건이다. 처음 보는 시기(시간순)에서는 불량 231건 중 45건을 잡고 18건만 유형까지 맞혔다.

> **현장에 하는 말** — "지금 쓰면(배율 1.0, 처음 보는 시기 기준) 하루 1,000샷에 경보 약 190건이 뜨고, 그중 실제 불량은 약 43건입니다. 그래서 지금 검사는 그대로 하시되, 모델이 의심한 샷부터 먼저 보시는 방식으로 몇 주 시험해 보자고 제안합니다. 그동안 실제로 몇 개를 맞혔는지 세어서, 검사를 줄여도 되는지 함께 판단하겠습니다. 놓침 1건을 헛경보 몇 건과 같게 볼지, 하루 몇 샷까지 검사할 수 있는지는 현장에서 정해 주셔야 합니다."

10. AI가 틀린 것 — 무엇을 물었고, 무엇을 검증해 채택 · 기각했나

Claude(Claude Code)로 코드 작성, 실험 설계, 결과 요약을 했다. 채택 기준: 같은 fold에서 짝지어 비교해 평균 +0.01 이상 **그리고** 대부분의 fold에서 승리.

#	물은 것	AI 조언	검증 방법 · 결과	판정
1	불량을 더 잡으려면?	불량 샘플을 SMOTE · 노이즈로 늘려라	무작위 5-fold × 3: 탐지율 48% → 62%, macro F1 −0.010 ~ −0.023	기각
2	약한 클래스를 살리려면?	기타불량 · 변형만 늘리거나 확률을 키워라	무작위 5-fold × 3: −0.003 ~ −0.014, 기타불량 F1 0.25 → 0.19	기각
3	판정 기준은?	정상 확률을 낮춰 불량 쪽으로 옮겨라	무작위 5-fold × 3 (KNN+RF+XGB 앙상블에서 측정): 0.9~0.4 모두 하락, 1.1~2.0도 하락 → 1.0이 최적	기각
4	클래스 가중치 효과는?	(AI가 만든 코드가) 가중치 적용 0.58 출력	가중치를 고른 데이터로 그대로 채점한 낙관적 값. 중첩 검증으로 다시 재니 0.546 (+0.006뿐)	**AI 계산 오류 발견**
5	test 샷을 학습에 쓰면?	확신 높은 test 샷에 예측 라벨을 붙여 추가	무작위 5-fold × 3: −0.001 ~ −0.005. 추가된 샷의 85~97%가 정상	기각
6	생산 기간 정보를 더 쓰려면?	KNN에서 Spray_1_Time 비중을 2~10배로	무작위 5-fold × 3: 예측이 하나도 안 바뀜. KNN 이웃의 99.7%가 이미 같은 Spray_1_Time	기각 (이미 반영됨)
7	변수를 조합하면?	비율 · 차이 · 곱 등 변수 51개 생성	중복 기록 묶기 5-fold × 3 (시드 3개): −0.006	기각
8	공정 지식 변수는?	용탕 − 냉각수 온도, 이슬점, 건조도 등 4개	중복 기록 묶기 5-fold × 3: RF +0.019 (15번 중 13번). KNN은 무작위 +0.0035 (15번 중 10번)로 기준 미달	원인 분석에 활용, 추가 실험 과제로 남김
9	MLP 시간순 결과가 이상하게 낮음	(AI 코드) 조기 종료용 15%를 학습 구간 끝에서 뗌	그 구간만 제품 2 시작이라 epoch 1에 멈춰 전부 한 클래스로 예측. 학습 구간 안에서 무작위로 떼도록 고침	**AI 코드 오류 발견 · 수정**
10	검증 방법은?	중복 기록을 같은 fold로 묶는 검증을 같이 봐라	KNN 무작위 0.562 vs 중복 기록 묶기 0.268	채택
11	(보고서 검토) 시간순 macro F1	시간순 검증 점수 0.246	검증 구간에 3개 클래스만 있어 7개가 아닌 3개로 평균한 값. 7개 기준으로 다시 계산 → 0.138 · 0.141	**평가 오류 발견 · 수정**

한계와 더 실험해 볼 부분

한계

- **처음 보는 시기에는 약하다.** 시간순 검증에서 불량 231건 중 45건(19%)만 잡았고, 어떤 경보 배율도 전수 검사보다 비용이 컸다(7장). 지금 기록된 21개 값으로는 같은 값이 학습 데이터에 없는 샷의 불량을 가르기 어렵다(0.27).
- **리더보드 점수는 중복 기록의 영향을 받는다.** test의 57.7%가 train에 같은 기록을 가져, public 0.589는 무작위 분할(0.570)과 비슷한 조건의 점수다. private도 같은 구조라면 비슷한 수준일 것으로 보지만, 아직 채점 전이다.
- **KNN 설정은 같은 train으로 골랐다.** 84개 조합 비교와 성능 평가에 같은 데이터를 써서 교차검증 점수가 약간 낙관적일 수 있다.
- **원인 분석은 관련성이다.** 냉각수 온도 · 압력은 생산 시기와 겹쳐, 조건 자체의 효과인지 그 시기의 다른 변화인지 가르지 못했다. 통제 시험 생산으로 확인해야 한다(8장).
- **운영표는 가정 위에 있다.** 놓침 1 = 헛경보 10, 하루 1,000샷, 전수 검사가 불량을 모두 잡는다는 가정이다. 현장 단가와 검사 가능 수량이 정해지면 다시 계산해야 한다.
- **AI 조언은 대부분 이 데이터에 맞지 않았다.** 11건 중 채택은 2건(8 · 10번)이고, AI가 만든 코드 · 평가 오류 3건(4 · 9 · 11번)은 결과를 직접 열어 보고 찾아 고쳤다.

더 실험해 보면 의미 있을 부분

- **① 이슬점 · 건조도 변수 추가.** 변형 · 기타불량은 건조도 중앙값이 12.5 · 11.9로 정상(7.9)보다 높았고, 이 변수들을 넣은 RF는 중복 기록 묶기 검증에서 15번 중 13번 이겼다(+0.019). 다만 KNN은 +0.0035(15번 중 10번), MLP는 다섯 번 중 두 번만 올라 효과가 일정하지 않아 이번 최종 모델에는 넣지 않았다. 지금까지는 용탕 − 냉각수 온도 · 용탕 − 공장 온도와 함께 4개를 한꺼번에 넣었으므로, **이슬점 · 건조도만 따로 넣어** 처음 보는 시기(시간순) 기준으로 다시 확인해 볼 만하다.
- **② 데이터 추가 — 양보다 종류.** 같은 기간의 기록을 5배로 늘려도 시간순 성능은 0.136 → 0.141로 거의 그대로였다(5장). 그래서 같은 기록을 더 모으기보다 지금 없는 값을 새로 재는 것이 먼저다. 예: **금형 온도**(지금은 용탕 − 냉각수 온도로 대신 짐작만 함), 그리고 설정을 바꾼 **다른 생산 시기의 기록**(변형은 생산 초반에만, 얼룩은 4구간에만 있어 시간순 검증 구간에는 3가지 클래스만 있었다).
- **③ 통제 시험 생산.** 냉각수 온도 · 변형 조건을 하나씩만 바꿔 생산해, 8장의 관련성이 원인인지 확인한다.